

EC2 : votre première machine dans le cloud

La VM Linux à la demande, par API

1. EC2 et les AMI

La machine virtuelle, à la demande

EC2 = votre VM du bloc Linux... à la demande

EC2 (Elastic Compute Cloud) fournit des **instances** : des machines virtuelles Linux ou Windows, facturées à l'usage.

Rien de magique : c'est la même chose que la VM Ubuntu du bloc CL-LINUX et du TP1 StockLine, avec trois différences fondamentales :

- **À la demande** : elle existe en ~40 secondes, elle disparaît en une commande. Plus d'hyperviseur à installer, plus d'ISO à télécharger.
- **Par API** : tout ce que fait la console est un appel d'API — donc scriptable. « Tout est API » (leitmotiv du J1).
- **Facturée à la seconde** : on paie ce qu'on consomme, pas ce qu'on possède.

Ce qui ne change PAS : une fois connecté, c'est un Ubuntu 24.04 ordinaire — `systemd`, `nginx`, `apt`, SSH avec clés. **Tous vos acquis Linux s'appliquent tels quels.**

Aujourd'hui : le VPC par défaut (et c'est voulu)

Une instance EC2 vit **toujours dans un réseau** : un **VPC** (Virtual Private Cloud) avec ses sous-réseaux — le plan d'adressage CIDR du bloc réseau, version AWS.

Point pédagogique important :

- AWS crée dans chaque région un **VPC par défaut** (CIDR 172.31.0.0/16, un sous-réseau public par AZ, une passerelle Internet) précisément pour qu'on puisse **démarrer sans rien construire**.
- **Aujourd'hui, nous l'utilisons tel quel**. Toutes nos instances y atterrissent automatiquement.

C'est une démarche classique d'apprentissage AWS : utiliser le défaut, puis le remplacer en connaissance de cause.

L'AMI : l'image de départ de l'instance

Une **AMI** (Amazon Machine Image) = le modèle à partir duquel l'instance est créée :

- **un système d'exploitation** (Ubuntu 24.04, Amazon Linux, Windows Server...),
- **des logiciels préinstallés** éventuels (une AMI peut embarquer nginx, Docker, une appli complète),
- le **contenu initial du disque racine** (un snapshot EBS, on y revient en section 6).

Analogie bloc Linux : l'AMI est à l'instance ce que l'ISO d'installation était à votre VM — sauf qu'ici le système est **déjà installé**, prêt à démarrer.

Nous utiliserons l'**AMI Ubuntu 24.04 LTS officielle de Canonical** (l'éditeur d'Ubuntu — la même distribution qu'en CL-LINUX).

⚠ Un ID d'AMI (`ami-0abc...`) est **différent dans chaque région** et **change à chaque mise à jour de l'image**.
Le copier en dur dans un script = script cassé tôt ou tard.

Trouver l'AMI proprement : le paramètre SSM public

Canonical publie l'ID de sa dernière AMI dans un **paramètre SSM public** (SSM Parameter Store : un service d'annuaire clé/valeur d'AWS). On l'interroge, on ne le devine pas :

```
AMI_ID=$(aws ssm get-parameters \
  --names /aws/service/canonical/ubuntu/server/24.04/stable/current/amd64/hvm/ebs-gp3/ami-id \
  --query 'Parameters[0].Value' --output text)

echo "$AMI_ID"
# ami-04a92520784b93e73 (exemple : la valeur évolue avec les mises à jour)
```

Lecture du chemin : `canonical` (éditeur) / `ubuntu server 24.04 stable` / `current` (toujours la dernière) / `amd64` (architecture) / `hvm/ebs-gp3` (virtualisation + type de disque racine).

Règle du cursus : jamais d'ID d'AMI en dur. Toujours résolu à l'exécution — c'est votre premier réflexe d'automatisation de la journée.

2. Familles et types d'instances

Choisir la bonne taille de machine

Anatomie d'un type d'instance

```

      t 3 . micro
      |   |
famille —|   |
(usage visé) |   |
t = burstable |   |
              |   |
      génération (3e) : plus récent = plus performant
                      et souvent MOINS cher

t3.micro  = 2 vCPU (en rafale), 1 Gio de RAM
m5.large = 2 vCPU (constants), 8 Gio de RAM
  
```

taille : nano < micro < small
< medium < large < xlarge < 2xlarge...
(à chaque cran : ~x2 vCPU et RAM)

Un **type d'instance** fige le matériel : vCPU, RAM, réseau. On ne « redimensionne » pas à chaud : on stoppe, on change le type, on redémarre.

Les familles, en survol

Famille	Profil	Cas d'usage type	Ordre de prix (eu-west-3)
t	Burstable, généraliste éco	Dev, test, petits sites, notre formation	t3.micro ≈ 0,012 \$/h
m	Généraliste équilibré (1 vCPU : 4 Gio)	Applis web de prod, StockLine en charge	m5.large ≈ 0,11 \$/h
c	Optimisé calcul (1 vCPU : 2 Gio)	Encodage, batch CPU, calcul scientifique	c5.large ≈ 0,10 \$/h
r	Optimisé mémoire (1 vCPU : 8 Gio)	Bases de données, caches en RAM	r5.large ≈ 0,15 \$/h
g	GPU	ML/inférence, rendu graphique	g4dn.xlarge ≈ 0,65 \$/h

Les familles, en survol

À retenir en tant qu'admin :

- On part du **besoin** (RAM ? CPU ? GPU ?), pas du catalogue.
- Il existe des centaines de types — connaître les **5 familles** suffit pour 95 % des choix.
- En formation : **t3.micro, toujours** (sauf mention contraire).

Le free-tier EC2 : 750 heures par mois

Le free-tier (12 premiers mois du compte) inclut :

- **750 h/mois** d'instance **t2.micro** ou **t3.micro** (selon la région ; à Paris les deux existent, on prend t3.micro, plus récente),
- 30 Go de stockage EBS, 1 Go de snapshots.

750 h $\approx$ 31 jours $\times$ 24 h : **une** instance peut tourner **en continu** tout le mois gratuitement.

“ 💰 **Coût — le piège des 750 h** : le quota est **global, pas par instance**. Deux t3.micro allumées en permanence = $\sim 1\,460$ h $\rightarrow$ ~ 710 h facturées $\approx$ **8,40 \$/mois**. Trois instances oubliées un week-end, ça se voit sur l'alerte de budget de 10 USD créée hier. D'où la règle : **teardown chaque soir**, on ne « garde pas pour demain ». ”

Vérifier ce qui tourne, à tout moment :

```
aws ec2 describe-instances \
  --filters "Name=instance-state-name,Values=running" \
  --query 'Reservations[].Instances[].[InstanceId,InstanceType,Tags[?Key==`Name`][0].Value]' \
  --output table
```

Instances burstables : les crédits CPU

La famille **t** est dite **burstable** (« en rafale ») : elle est peu chère parce qu'elle ne garantit pas 100 % du CPU en continu.

- Chaque instance a un **niveau de base** (baseline) : ~10 % d'un vCPU pour une t3.micro.
- Sous la baseline, elle **accumule des crédits CPU** ; au-dessus, elle les **dépense**.
- Crédits épuisés, deux comportements :
 - **mode standard** : le CPU est **bridé** à la baseline (l'appli rame, symptôme classique) ;
 - **mode unlimited** (défaut des t3) : le CPU continue... mais le dépassement est **facturé** (~0,05 \$ par vCPU-heure).

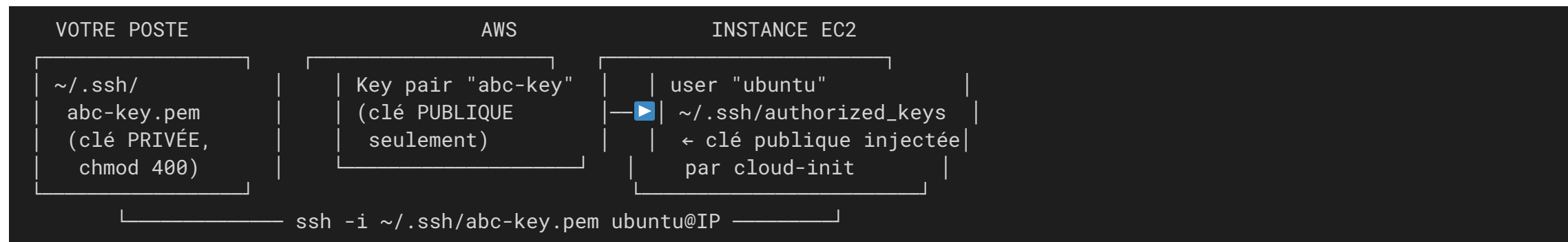
Conséquences pratiques :

- Parfait pour dev/test/formation : charge irrégulière, longues périodes calmes.
- **À proscrire** pour une charge CPU soutenue (encodage 24/7) : soit ça rame, soit ça coûte plus cher qu'une famille c.
- Métrique à surveiller : `CPUCreditBalance`.

3. Key pairs, SSH et security group

Vos réflexes du bloc Linux, version cloud

La key pair : vos clés SSH, gérées par AWS



Le mécanisme LINUX (`ssh-keygen`, `authorized_keys`) — AWS se charge juste de **livrer la clé publique** dans l'instance au premier boot. AWS **ne conserve jamais la clé privée** : perdue = pas de récupération.

Créer sa key pair à la CLI

```
aws ec2 create-key-pair \  
  --key-name "$PREFIX-key" \  
  --key-type ed25519 \  
  --query 'KeyMaterial' --output text > ~/.ssh/$PREFIX-key.pem  
  
chmod 400 ~/.ssh/$PREFIX-key.pem      # réflexe CL-LINUX : clé privée illisible par les autres
```

Points d'attention :

- `--key-type ed25519` : l'algorithme moderne (par défaut AWS génère du RSA).
- `--query 'KeyMaterial' --output text` : extrait **la clé privée**, affichée **une seule fois** — d'où la redirection immédiate dans un fichier.
- Sans le `chmod 400`, SSH refusera la clé : `WARNING: UNPROTECTED PRIVATE KEY FILE!`

Vérifier :

```
aws ec2 describe-key-pairs --key-names "$PREFIX-key" \  
  --query 'KeyPairs[0].[KeyName,KeyType]' --output text  
# abc-key    ed25519
```

Le security group d'accès : 22 pour vous, 80 pour tous

Rappel réseau : un **security group** est un pare-feu **stateful** attaché à l'instance (la réponse à un flux autorisé passe toujours – contrairement aux NACL stateless).

Exemple SG , dans le VPC par défaut :

```
SG_ID=$(aws ec2 create-security-group \
  --group-name "$PREFIX-sg-web" \
  --description "Web + SSH restreint - formation J2" \
  --query 'GroupId' --output text)

MON_IP=$(curl -s https://checkip.amazonaws.com) # votre IP publique actuelle

aws ec2 authorize-security-group-ingress --group-id "$SG_ID" \
  --protocol tcp --port 22 --cidr "${MON_IP}/32" # SSH : VOUS, et personne d'autre
aws ec2 authorize-security-group-ingress --group-id "$SG_ID" \
  --protocol tcp --port 80 --cidr "0.0.0.0/0" # HTTP : public, c'est un site web
```

 **Jamais** `0.0.0.0/0` **sur le port 22**. Une instance SSH ouverte au monde est scannée en quelques **minutes** (regardez `/var/log/auth.log`). `/32` = exactement une adresse – votre notation CIDR du bloc réseau.

4. Lancement et user data

Console d'abord, CLI ensuite — puis la machine s'installe seule

Lancer depuis la console : l'assistant, écran par écran

Console → EC2 → **Launch instance**. L'assistant tient en 7 blocs, tous à comprendre :

1. **Name and tags** : `$PREFIX-web-console` → devient le tag `Name`.
2. **Application and OS Images (AMI)** : Ubuntu Server 24.04 LTS — vérifiez l'éditeur **Canonical** et la mention *Free tier eligible*.
3. **Instance type** : `t3.micro` (*Free tier eligible*).
4. **Key pair** : `$PREFIX-key` — celle créée à la CLI (la preuve que console et CLI voient les mêmes objets : **tout est API**).
5. **Network settings** : VPC par défaut, *Auto-assign public IP: Enable*, **Select existing security group** → `$PREFIX-sg-web`.
6. **Configure storage** : 8 Gio **gp3** — le volume racine EBS (section 6).
7. **Advanced details** : tout en bas, **User data** (section suivante) et **Metadata version : V2 only (token required)**.

L'assistant n'est qu'un **formulaire qui construit un appel d'API**.

La même chose à la CLI : `run-instances`

```
INSTANCE_ID=$(aws ec2 run-instances \
  --image-id "$AMI_ID" \
  --instance-type t3.micro \
  --key-name "$PREFIX-key" \
  --security-group-ids "$SG_ID" \
  --user-data file:///code/06-cl-aws1/user-data-nginx.sh \
  --metadata-options "HttpTokens=required,HttpEndpoint=enabled" \
  --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=$PREFIX-web-cli}]" \
  --count 1 \
  --query 'Instances[0].InstanceId' --output text)

aws ec2 wait instance-running --instance-ids "$INSTANCE_ID" # bloque jusqu'à l'état running

aws ec2 describe-instances --instance-ids "$INSTANCE_ID" \
  --query 'Reservations[0].Instances[0].PublicIpAddress' --output text
# 51.44.112.87 (exemple)
```

Chaque option = un écran de l'assistant console. **Tout est API** : ce que la souris faisait en 3 minutes, la CLI le fait en 1 commande — donc un script peut le faire, donc c'est automatisable. C'est ça, le métier. Connexion, avec vos réflexes CL-LINUX (utilisateur `ubuntu` sur les AMI Canonical) :

```
ssh -i ~/.ssh/$PREFIX-key.pem ubuntu@51.44.112.87
```

User data : la machine s'installe toute seule

Le **user data** est un script fourni **au lancement**, exécuté par **cloud-init** (outil standard des images cloud Ubuntu) :

- **une seule fois**, au **premier démarrage** (pas aux reboots),
- en **root** (pas de `sudo` dans le script),
- **après** le réseau — l'instance peut donc télécharger des paquets.

5. Métadonnées et IMDSv2

**La carte d'identité de l'instance — et
pourquoi on la protège**

169.254.169.254 : l'instance se renseigne sur elle-même

Depuis **l'intérieur** d'une instance (et seulement de là), l'adresse `169.254.169.254` sert les **métadonnées d'instance** (IMDS – Instance Metadata Service) :

- `instance-id`, `instance-type`, `ami-id`,
- `placement/availability-zone`, `local-ipv4`, `public-ipv4`,
- `security-groups`, `tags`, et surtout... les **identifiants temporaires IAM** si un rôle est attaché (J4).

Rappel bloc réseau : `169.254.0.0/16` est la plage **link-local** – non routable, jamais transmise par un routeur. C'est pour ça que l'adresse est la même sur toutes les instances du monde.

À quoi ça sert ? C'est ce qui permet à un script de **s'adapter à la machine où il tourne** sans rien coder en dur : « quel est mon ID ? ma zone ? mon IP ? ». La brique de base de toute automatisation EC2.

IMDSv2 : pourquoi un jeton obligatoire (l'attaque SSRF)

IMDSv1 (dangereux)

Attaquant → votre appli web
"va chercher cette URL :"
`http://169.254.169.254/...`



L'appli (DANS l'instance) obéit,
un simple GET suffit →
identifiants IAM exfiltrés ✨

IMDSv2 (imposé aujourd'hui)

- 1) PUT /latest/api/token
→ jeton de session (TTL ≤ 6 h)
- 2) GET /meta-data/...
+ header X-aws-ec2-metadata-token

Un simple GET forgé par SSRF
ne peut PAS faire le PUT préalable
→ requête sans jeton = 401 🙅

SSRF (Server-Side Request Forgery) : faire faire une requête HTTP par le serveur à votre place. Avec IMDSv1, un simple GET volait les identifiants du rôle (cause de la fuite Capital One, 2019 – 100 M de clients).

IMDSv2 en pratique : imposer, puis interroger

Imposer à la création — nos lancements du jour le font déjà :

```
--metadata-options "HttpTokens=required,HttpEndpoint=enabled"
```

Interroger, depuis l'instance (le motif utilisé par notre user data) :

```
# 1. Obtenir un jeton de session (méthode PUT, TTL en secondes, max 21600 = 6 h)
TOKEN=$(curl -sX PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

# 2. Interroger en présentant le jeton
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/instance-id
# i-0a1b2c3d4e5f67890
```

Sans jeton, quand `HttpTokens=required` :

```
curl -s -o /dev/null -w '%{http_code}\n' http://169.254.169.254/latest/meta-data/
# 401
```

Réflexe d'admin cloud : **IMDSv2 required sur toute instance, toujours**. C'est le « sécuriser » du fil conducteur — dès le jour 2.

6. EBS et IP élastiques

Des disques qui survivent, des adresses qui ne bougent pas

EBS : les disques de vos instances

EBS (Elastic Block Store) fournit des **volumes** : des disques réseau attachés aux instances (vus comme `/dev/nvme0n1` – vos réflexes `lsblk/df -h` du bloc Linux s'appliquent).

Type	Usage	À retenir
gp3	SSD généraliste – le défaut	3 000 IOPS de base, taille et IOPS réglables séparément
gp2	SSD ancienne génération	IOPS liés à la taille – encore croisé, à migrer vers gp3
io1 / io2	SSD hautes performances	Bases de données exigeantes, IOPS provisionnés, cher
st1 / sc1	Magnétique débit / froid	Gros volumes séquentiels, archives – pas pour un OS

Le **volume racine** : créé depuis l'AMI au lancement (8 Gio gp3 pour notre Ubuntu), il contient l'OS. Par défaut, il est **supprimé avec l'instance** (`DeleteOnTermination=true` pour le racine) – les volumes ajoutés en plus, eux, survivent par défaut.

Ordre de prix : gp3 ≈ **0,09 \$/Gio/mois** à Paris → nos 8 Gio ≈ 0,75 \$/mois. Petit, mais **facturé même instance éteinte** (slide 💰 en section 7).

Snapshots : la sauvegarde incrémentale des volumes

Un **snapshot** = photo d'un volume EBS à l'instant T, stockée **dans S3 en coulisse** (vous ne voyez pas le bucket, AWS le gère) :

- **Incrémental** : le premier copie tout, les suivants ne copient que les **blocs modifiés** – rapides et peu coûteux (~0,05 \$/Gio/mois, sur les blocs réellement stockés).
- Un snapshot est **régional** (il survit à la perte de l'AZ du volume) et **copiable** vers une autre région.
- C'est aussi la matière première des **AMI personnalisées** : AMI = snapshot(s) + métadonnées de démarrage.

```
# Créer un snapshot du volume, puis attendre qu'il soit terminé
SNAP_ID=$(aws ec2 create-snapshot --volume-id "$VOL_ID" \
  --description "$PREFIX - racine web - J2" \
  --query 'SnapshotId' --output text)
aws ec2 wait snapshot-completed --snapshot-ids "$SNAP_ID"

# Restaurer = créer un NOUVEAU volume à partir du snapshot (dans l'AZ voulue)
aws ec2 create-volume --snapshot-id "$SNAP_ID" \
  --availability-zone eu-west-3a --volume-type gp3
```

Restaurer ne modifie jamais l'original : on crée un volume neuf, qu'on attache où on veut.

IP élastiques : une adresse publique qui ne bouge plus

Constat (démontré tout à l'heure) : l'**IP publique classique** d'une instance est **rendue au stop** et **différente au start**. Gênant pour un DNS, un pare-feu distant, un client...

L'**Elastic IP (EIP)** est une IP publique **allouée à votre compte**, que vous associez/dissociez à volonté : elle **survit aux stop/start** de l'instance.

```
ALLOC_ID=$(aws ec2 allocate-address --query 'AllocationId' --output text)
aws ec2 associate-address --instance-id "$INSTANCE_ID" --allocation-id "$ALLOC_ID"
# ...et pour rendre l'adresse :
aws ec2 release-address --allocation-id "$ALLOC_ID"
```

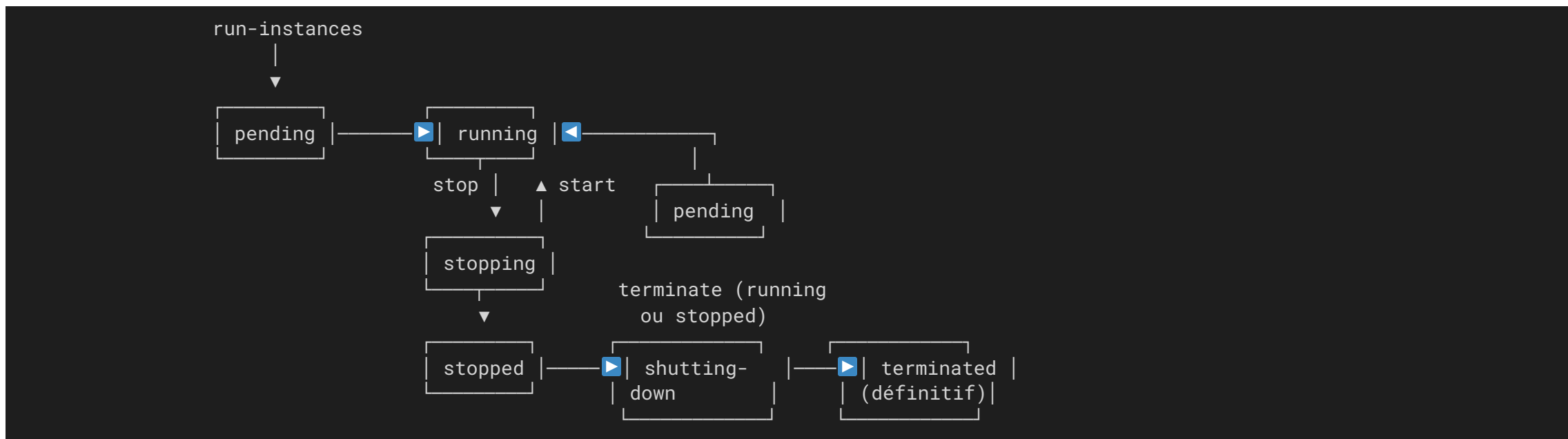
“ 💰 **Coût** — l'EIP se paie dès qu'elle est allouée : **0,005 \$/h ≈ 3,60 \$/mois**, qu'elle soit associée ou non, instance allumée ou pas. Les IPv4 publiques sont une ressource rare, AWS facture leur détention. Quota : **5 EIP par région** par défaut. Règle du cursus : **on libère (release-address) dès qu'on n'en a plus besoin** — une EIP « orpheline » est le grand classique de la facture surprise. ”

En prod on préfère souvent un nom DNS stable (load balancer — bloc 08) à une EIP. L'EIP reste utile pour un serveur unique à adresse fixe.

7. Cycle de vie et facturation

**Savoir ce qui coûte, quand — et tout éteindre
proprement**

Le cycle de vie d'une instance



stopped : redémarrable, disque conservé. **terminated** : **irréversible**, volume racine supprimé (par défaut).

Ce que vous payez, état par état

Facturation du **calcul** EC2 : à la **seconde** (minimum 60 s), uniquement à l'état **running**. Une instance stoppée ne coûte **rien en calcul**. Mais l'instance n'est pas seule :

Ressource	running	stopped	terminated
Calcul (vCPU/RAM)	✓ facturé (à la seconde)	✗	✗
Volume EBS	✓ facturé	✓ facturé quand même	✗ (racine supprimé)
EIP allouée	✓ 0,005 \$/h	✓ 0,005 \$/h	✓ tant que non libérée !
Snapshots	✓	✓	✓ tant que non supprimés

“ 💰 **Coût** — « **stoppée** » ne veut pas dire « **gratuite** » : le volume EBS ($\approx 0,09$ \$/Gio/mois) et l'EIP continuent de compter. Et un snapshot ou une EIP **survivent au terminate** de l'instance. La facture ne s'arrête que quand **chaque** ressource est détruite ou libérée. ”
D'où la discipline : le teardown se vérifie ressource par ressource, pas « à l'œil ».

Stop vs terminate — et le teardown du soir

- **stop-instances** : pause. On garde le disque, les tags, l'ID. On paie l'EBS. L'IP publique classique sera **perdue**.
- **terminate-instances** : destruction **définitive**. Volume racine supprimé (défaut), ID irrécupérable. C'est le geste de fin de journée.

Le rituel du soir (dans l'ordre — celui de la démo) :

```
aws ec2 terminate-instances --instance-ids "$INSTANCE_ID"
aws ec2 wait instance-terminated --instance-ids "$INSTANCE_ID"
aws ec2 release-address --allocation-id "$ALLOC_ID"          # EIP
aws ec2 delete-snapshot --snapshot-id "$SNAP_ID"             # snapshots
aws ec2 delete-security-group --group-id "$SG_ID"            # SG (après terminate !)
aws ec2 delete-key-pair --key-name "$PREFIX-key" && rm ~/.ssh/$PREFIX-key.pem
```

Puis on **vérifie** (instances, adresses, volumes, snapshots) — les commandes de contrôle sont dans la démo et l'exercice 2-2. Un SG ne se supprime pas tant qu'une instance l'utilise : d'où le **wait**.